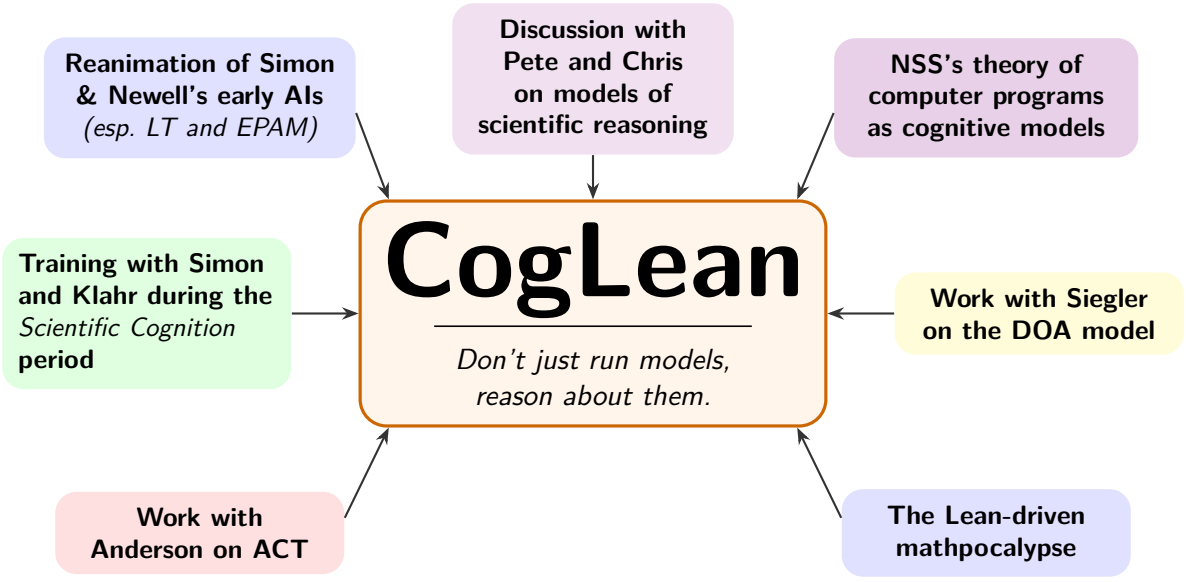
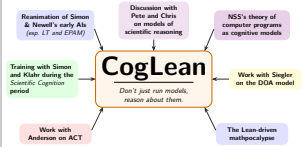
Epoofany

Epoofany

Epoofany.


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

CogLean



Where CogLean came from.

CogLean

Towards a Cumulative, Machine-Checked Theory of Cognition

Jeff Shrager

October 2026

Draft for Comment

CogLean



This talk goes with the paper of the same name, which is a draft for comment. It hasn't been carefully checked yet: think of it as an over-starched blog post, and constructive feedback is welcome. The paper introduces CogLean, a library of cognitive theory in the Lean proof assistant, and argues for a way of doing theoretical cognitive science: theories stated once, their consequences proved, and independently developed theories composed so that new results follow from old ones. I'm going to tell it through a hypothetical modeler, Chris, who uses the library to answer two questions about children's arithmetic: one forward, from a phenomenon to a theorem, and one backward, from a proposed regularity to what a theory must be like to produce it. The central example is multi-digit addition, but I want to be clear from the start that it is a demonstration, not a new theory of arithmetic. And one more thing up front: this is less about proof, in the usual sense, than about explanation. The theorems are explanations of phenomena, and Lean keeps them aligned with the programs they explain.

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable *as a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable as *a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?

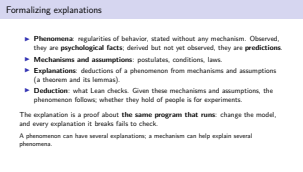
The problem is an old one, and the people who have done the most about it are the architecture builders. Newell complained in 1973 that we play twenty questions with nature and never assemble the answers, and his answer was unified theories of cognition. Soar and ACT-R, and now the Common Model of Cognition, are decades of work by Newell, Anderson, Laird, Rosenbloom and many others on exactly this: shared mechanisms and constraints, so that a model of a new task reuses the architecture's theory of memory, learning and control, and models of different tasks can be integrated. CogLean is not the first effort in this direction, and it isn't a competitor. What architectures accumulate, though, is mainly models. What a model predicts is established by running it, at one setting at a time, and a result established for one model is rarely available as a result to another. CogLean pursues a complementary goal: accumulating machine-checked theorems about cognitive mechanisms, which can be reused deductively, and stated independently of any one architecture's commitments, so they can cover classes of mechanisms. An architecture could, in principle, carry such results itself. The question for this talk is whether a common formal library can change that: whether theories in such a library can be composed to give meaningful

Formalizing explanations

- **Phenomena**: regularities of behavior, stated without any mechanism. Observed, they are **psychological facts**; derived but not yet observed, they are **predictions**.
- **Mechanisms and assumptions**: postulates, conditions, laws.
- **Explanations**: deductions of a phenomenon from mechanisms and assumptions (a theorem and its lemmas).
- **Deduction**: what Lean checks. Given these mechanisms and assumptions, the phenomenon follows; whether they hold of people is for experiments.

The explanation is a proof about **the same program that runs**: change the model, and every explanation it breaks fails to check.

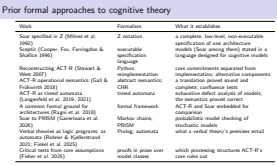
A phenomenon can have several explanations; a mechanism can help explain several phenomena.



Before any Lean, here's what this is for. It's less about proof, in the usual sense, than about explanation. What we explain are phenomena, stated without any mechanism: observed, they're psychological facts; derived but not yet observed, they're predictions. Underneath are mechanisms and their assumptions: postulates, conditions and laws. An explanation is a deduction of a phenomenon from those, in Lean a theorem and its chain of lemmas. That's Hempel and Oppenheim's old deductive picture of explanation, running through how the mechanisms work, made checkable. Lean proves conditional statements; whether the assumptions hold of people is for experiments. And here's what I think matters most. Usually a model is a program and its explanation lives somewhere else, in prose or in math about an idealized program, and the two drift apart. In Lean, the definitions that run are the ones the theorems are about, so when the model changes, every explanation it breaks fails to check, and the rest are confirmed. Not every proof is an explanation; that stays a judgment. But one phenomenon can have several explanations, and one mechanism can help explain several phenomena, and a library of them lets us ask what observations would tell explanations apart.

Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of one architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Frühwirth 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar to PRISM (Ganeriwala et al. 2026)	Markov chains, PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rohner & Kjellerstrand 2021; Finkel et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out


Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of core architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Fridsma 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar in PRISM (Gluck & Laird 2018)	Markov chains; PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rubow & Kjellerstrand 2021; Fisher et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out

CogLean is far from the first formal treatment of cognitive theory, and it's worth seeing what came before. In 1992 Milnes and colleagues specified Soar in the Z notation: complete, but low-level and not executable. Cooper, Fox, Farringdon and Shallice argued in 1996 for languages designed for specifying cognitive models, and built one, Sceptic. Stewart and West rebuilt ACT-R to separate its core commitments from its implementation. For ACT-R there's a formal operational semantics with a translation proved sound and complete, and a line of work translating ACT-R models into timed automata for exhaustive defect checking. Ragni and colleagues proposed a common formal ground into which ACT-R and Soar can be embedded. Most recently, Soar models have been compiled for the PRISM probabilistic model checker. Outside architectures, verbal theories have been formalized as logic programs and as automata, and Fisher and colleagues derived critical tests of ACT-R from its core assumptions. Almost all of this work formalizes one architecture or one model at a time, and establishes properties of that system. That's the contrast I'll come back to.

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

Mathematics faced a version of this problem and has made real progress on it. Mathlib, the community library for the Lean proof assistant, doesn't propose new mathematics. It's a shared environment: definitions are made once, every result is checked by machine, and later work builds on earlier results by citing them. A theorem about groups, once proved, is available to everyone who works with groups. CogLean asks whether cognitive theory can be organized the same way. It is not a new theory of cognition, and it's not a new modelling language either. And it's nowhere near Mathlib's scale; this is a first demonstration that the approach can work.

CogLean's layers

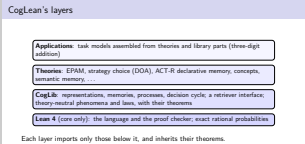
Applications: task models assembled from theories and library parts (three-digit addition)

Theories: EPAM, strategy choice (DOA), ACT-R declarative memory, concepts, semantic memory, ...

CogLib: representations, memories, processes, decision cycle; a retriever interface; theory-neutral phenomena and laws, with their theorems

Lean 4 (core only): the language and the proof checker; exact rational probabilities

Each layer imports only those below it, and inherits their theorems.



CogLean has four layers. At the bottom is Lean, core only, with exact rational probabilities. Above it is CogLib, the general library: representations such as feature codes and distributions; memory structures such as discrimination nets and an activation memory; processes such as sampled retrieval and a decision cycle with impasses; an interface that any memory can implement; and theory-neutral statements of phenomena and laws, with theorems about them. Above that are the theories, each stated as choices of library mechanisms. And at the top are applications, models of particular tasks assembled from theories and library parts. Each layer imports only those below it, and inherits their theorems.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Four kinds of object have to be kept apart. Mechanisms and their postulates are definitions. Phenomena are predicates, stated without reference to any mechanism; a predicate asserts nothing, it's a target. Theorems say that a mechanism produces behaviour satisfying a predicate. And conditions are the hypotheses under which a theorem holds: some situational, some empirical premises we haven't explained yet, some just well-formedness. One rule follows: a theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction of any model. The library's catalog flags those mechanically.

Scientific notions, and what they are in Lean

Model (program)	executable definitions: the same ones that run
Mechanism	a CogLib process, chosen by a theory
Phenomenon	a predicate, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: *outside Lean*
Prediction	a theorem's conclusion, not yet observed
Condition	a theorem's hypothesis
Law	a property of a mechanism: proved for a family, or required by a class theorem
Explanation	a theorem and its lemmas
Deduction	what Lean's kernel checks
Illustration	a value computed at one setting: a witness, not an explanation
Lean's axioms	logic only, never scientific assumptions

Scientific notions, and what they are in Lean

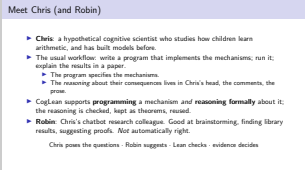
Model (program)	executable definitions: the same ones that run
Mechanism	a CogLib process, chosen by a theory
Phenomenon	a predicate, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: outside *Lean*
Prediction	a theorem's conclusion, not yet observed
Condition	a theorem's hypothesis
Law	a property of a mechanism: proved for a family, or required by a class theorem
Explanation	a theorem and its lemmas
Deduction	what Lean's kernel checks
Illustration	a value computed at one setting: a witness, not an explanation
Lean's axioms	logic only, never scientific assumptions

To be completely explicit about how the words line up. The model, the program, is a set of executable definitions, and they're the same definitions that get run. A mechanism is a CogLib process that a theory chooses. A phenomenon is a predicate stated with no mechanism in it; it's a target. A psychological fact is a phenomenon that holds of people, and that judgment is outside Lean altogether. A prediction is a theorem's conclusion about something nobody has observed yet. Conditions are a theorem's hypotheses. A law is a property of a mechanism, either proved for a whole family or required by a theorem about a class. None of these are Lean axioms; a law of memory is a hypothesis in the same way the group axioms are hypotheses in Mathlib. An explanation is a theorem and its chain of lemmas, and deduction is what Lean's kernel checks. A number computed at one setting is an illustration, a witness that the hypotheses can be met, not an explanation. And Lean's own three axioms are just logic.

Meet Chris (and Robin)

- **Chris**: a hypothetical cognitive scientist who studies how children learn arithmetic, and has built models before.
- The usual workflow: write a program that implements the mechanisms; run it; explain the results in a paper.
  - The program specifies the mechanisms.
  - The *reasoning* about their consequences lives in Chris's head, the comments, the prose.
- CogLean supports **programming** a mechanism *and* **reasoning formally** about it; the reasoning is checked, kept as theorems, reused.
- **Robin**: Chris's chatbot research colleague. Good at brainstorming, finding library results, suggesting proofs. *Not* automatically right.

Chris poses the questions · Robin suggests · Lean checks · evidence decides



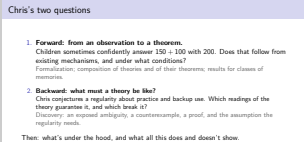
Let me introduce Chris. Chris is a hypothetical cognitive scientist who studies how children learn arithmetic, and has built computational models before. Like most of us, Chris is used to a particular workflow: write a program that implements the proposed mechanisms, run it to see what it does, and explain the results in a paper. The program specifies the mechanisms, but the reasoning about what they imply lives in Chris's head, in the comments, or in the prose. CogLean offers another way to work. It supports programming a mechanism and also reasoning formally about it, and that reasoning is checked by the computer, kept as theorems, and reusable in other models. Chris also has a colleague, Robin, who is a chatbot. Robin is genuinely useful: for brainstorming, for finding what's already in the library, for suggesting proofs. But Robin is not automatically right, and you'll see Robin be wrong. The division of labor is: Chris poses the questions, Robin suggests, Lean checks, and evidence decides which premises describe children.

Chris's two questions

1. **Forward: from an observation to a theorem.**
   Children sometimes confidently answer $150 + 100$ with 200. Does that follow from existing mechanisms, and under what conditions?
   Formalization; composition of theories and of their theorems; results for classes of memories.

2. **Backward: what must a theory be like?**
   Chris conjectures a regularity about practice and backup use. Which readings of the theory guarantee it, and which break it?
   Discovery: an exposed ambiguity, a counterexample, a proof, and the assumption the regularity needs.

Then: what's under the hood, and what all this does and doesn't show.



Chris has two questions, and they're the two halves of the talk. The first goes forward, from an observation to a theorem. Children sometimes confidently answer one fifty plus one hundred with two hundred. Does that follow from mechanisms we already have, and under exactly what conditions? Answering it uses formalization, composition of independently formalized theories and of their theorems, and results that hold for whole classes of memories. The second goes backward. Chris conjectures a regularity about practice and backup use, and asks what a theory has to be like to guarantee it. That one exposes an ambiguity in the theory, produces a counterexample and a proof, and isolates the assumption the regularity needs. After that I'll open the hood on the composition, and finish with what all this does and doesn't show.

Part 1

From an observation to a theorem

Does confident error follow from mechanisms we already have?

Part one. Chris's first question, going forward: does a familiar phenomenon, confident errors, follow from mechanisms already in the library?

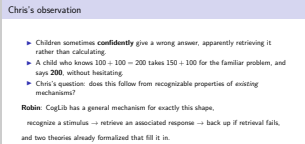
Chris's observation

- Children sometimes **confidently** give a wrong answer, apparently retrieving it rather than calculating.
- A child who knows $100 + 100 = 200$ takes $150 + 100$ for the familiar problem, and says **200**, without hesitating.
- Chris's question: does this follow from recognizable properties of *existing* mechanisms?

Robin: CogLib has a general mechanism for exactly this shape,

recognize a stimulus $\rightarrow$ retrieve an associated response $\rightarrow$ back up if retrieval fails,

and two theories already formalized that fill it in.



Here's Chris's observation. Children sometimes give a wrong answer confidently: they seem to be retrieving it, not calculating it. A child who knows one hundred plus one hundred is two hundred takes one fifty plus one hundred for the familiar problem, and says two hundred without hesitating. Chris's question is whether that follows from recognizable properties of mechanisms we already have, rather than from something invented for the purpose. Robin points out that the library has a general mechanism of exactly this shape: recognize a stimulus, retrieve a response associated with what was recognized, and fall back on a backup process if retrieval fails. And two existing theories, already formalized on their own, fill it in.

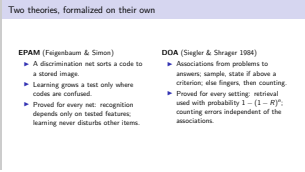
Two theories, formalized on their own

EPAM (Feigenbaum & Simon)

- A discrimination net sorts a code to a stored image.
- Learning grows a test only where codes are confused.
- Proved for every net: recognition depends only on tested features; learning never disturbs other items.

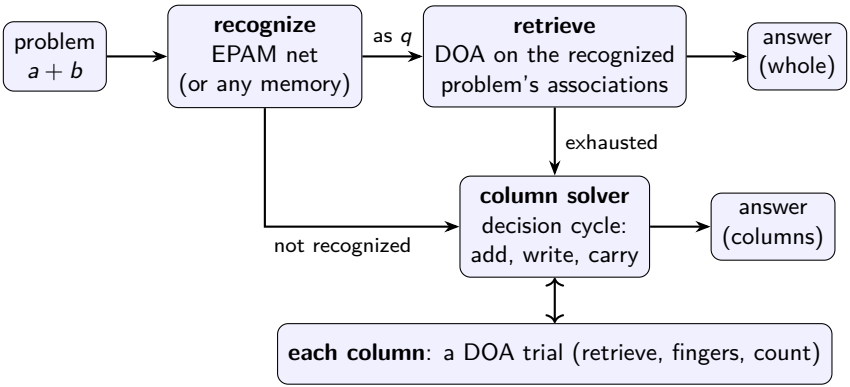
DOA (Siegler & Shrager 1984)

- Associations from problems to answers; sample, state if above a criterion; else fingers, then counting.
- Proved for every setting: retrieval used with probability $1 - (1 - R)^n$; counting errors independent of the associations.



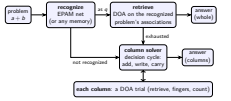
The two theories Robin has in mind were each formalized on their own, long before Chris's question. EPAM, Feigenbaum and Simon's model, recognizes a stimulus by sorting its code down a discrimination net; learning grows a test only where two codes are confused. Proved for every net: recognition depends only on the features tested, and learning never disturbs what's already learned. And the distributions-of-associations model that Bob Siegler and I proposed in 1984: a child samples answers in proportion to strength, states one that clears a confidence criterion, and otherwise falls back on fingers and counting. Proved for every setting: retrieval is used with probability one minus one minus R to the n, and counting errors don't depend on the associations at all, which was the chapter's nonintuitive prediction.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

Chris composes them into a model of three-digit addition. A problem is recognized by an EPAM net, or by any memory. If it's recognized as a familiar problem q, the answer is retrieved from q's distribution of associations. If retrieval is exhausted, or nothing is recognized, a column solver takes over. It runs on the library's decision cycle, adding, writing and carrying, and each column's addition is itself a full single-digit trial from the strategy-choice model. In Lean the whole composition is three definitions. Again: arithmetic is the demonstration. It's a familiar task that needs several mechanisms, and two of the theories already existed.

The phenomenon, stated without a mechanism, and when it follows

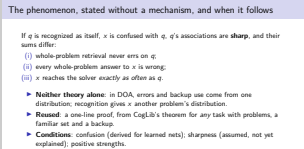
If q is recognized as itself, x is confused with q, q's associations are **sharp**, and their sums differ:

(i) whole-problem retrieval never errs on q;

(ii) every whole-problem answer to x is wrong;

(iii) x reaches the solver *exactly as often* as q.

- **Neither theory alone**: in DOA, errors and backup use come from one distribution; recognition gives x another problem's distribution.
- **Reused**: a one-line proof, from CogLib's theorem for *any* task with problems, a familiar set and a backup.
- **Conditions**: confusion (derived for learned nets); sharpness (assumed, not yet explained); positive strengths.

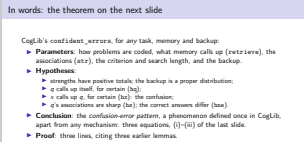


Chris and Robin first state the phenomenon without mentioning EPAM or DOA: a confused problem gets wrong answers from whole-problem retrieval, but uses its backup exactly as often as the problem it's confused with. Then they look for the conditions under which it follows. Suppose a familiar problem q is recognized as itself, a stranger x is confused with it, q's associations are sharp, meaning only q's correct sum clears the criterion, and the two sums differ. Then retrieval never errs on q, every whole-problem answer to x is wrong, and x goes to the solver exactly as often as q does. Neither theory says this alone: in strategy choice, a problem's errors and its backup use come from the same distribution, so they go together. Recognition breaks the tie by handing x another problem's distribution. The proof is one line, applying a library theorem that holds for any task with problems, a familiar set and a backup. And the conditions are explicit: confusion, which is derived for learned nets; sharpness, which is assumed and not yet explained; and positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.



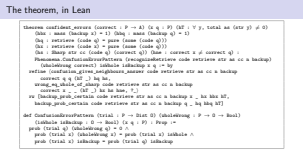
The next slide shows the actual Lean theorem, so here it is in words first. It's the library's version, which holds for any task, any memory and any backup. Its parameters are how problems are coded, what memory calls up for a code, the associations, the criterion and search length, and the backup. Its hypotheses: well-formedness of the strengths and the backup; the familiar problem calls up itself for certain; the stranger calls up the familiar problem for certain, which is the confusion; the familiar problem's associations are sharp; and the correct answers differ. The conclusion is the confusion-error pattern, which the library defines once, as a phenomenon, with no mechanism in it: the three equations from the previous slide. And the proof is three lines, each citing an earlier lemma.

The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

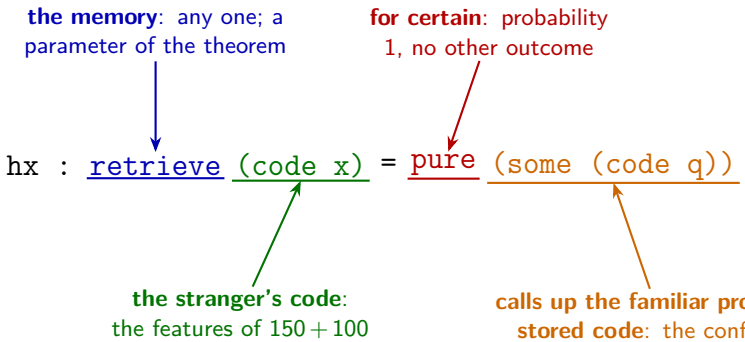
Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). The multi-digit version is a one-line instance.



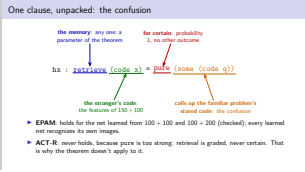
Verbatim from CogLib (Process/RecognizeRetrieve, Phenomena/ConfusionErrors). The multi-digit version is a one-line instance.

Here's the theorem itself, verbatim from the library. The first five lines are the hypotheses, each named: h T, the two backup hypotheses, h q, h x, h s and h n e. Then the conclusion: the trial has the confusion-error pattern. Then the proof. At the bottom is that pattern's definition, three equations joined by and, about observable outcomes only. The first clause is the lemma that a problem recognized as a sharp q gets only q's answer; the second is the lemma that all its whole-problem answers are then wrong; and the third rewrites both backup probabilities with the lemma that backup use after certain recognition depends only on what's recognized. Nothing in the statement mentions arithmetic, EPAM or ACT-R. The multi-digit model's theorem is a one-line instance of this one.

One clause, unpacked: the confusion



- **EPAM**: holds for the net learned from $100 + 100$ and $100 + 200$ (checked); every learned net recognizes its own images.
- **ACT-R**: *never* holds, because `pure` is too strong: retrieval is graded, never certain. That is why the theorem doesn't apply to it.



Let's unpack one clause, the confusion, h x. Read it left to right. Retrieve is the memory, and it's a parameter: the theorem holds for any memory you plug in. Code x is the stranger's feature code, here the features of one fifty plus one hundred. Pure means for certain: the distribution with a single outcome, at probability one. And some code q says the outcome is the familiar problem's stored code. So the clause says: this memory calls up the familiar problem, for certain, when given the stranger. For EPAM it holds: the net learned from one hundred plus one hundred and one hundred plus two hundred is checked to do exactly this. For ACT-R it never holds, and the culprit is the word pure: ACT-R's retrieval is graded and never certain. That single word is why the theorem applies to one memory and not the other.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

One precision matters here. The theorem says the confused problem reaches the solver exactly as often as the familiar one. It doesn't say that's rare. The rate is one minus R q to the n, where R q is the chance that one search of q's associations clears the criterion; and if R q is at least r, solver use is at most one minus r to the n. Both are proved. So whether these errors are confident, fluent and unhesitating, is a separate, quantitative matter. With flat associations the same theorem holds, but the errors wouldn't be confident. In our setting R q is forty-five over forty-six, so the solver is reached once in forty-six.

Does it need EPAM? Swapping the memory

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$: solver invoked	$1/46$	$383/1058 \approx .36$
solver correct, *given* invoked	$\approx .656$	$\approx .656$
correct overall (invoked *and* correct)	$\approx .014$	$\approx .238$
practiced $100 + 100$: overall errors	$\approx .0073$	$\approx .334$
after also studying $100 + 300$	unchanged	$\approx .616$
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).

Does it need EPAM? Swapping the memory

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	45/46	225/529 ≈ .43
$150 + 100$: solver invoked	1/46	383/1058 ≈ .36
solver correct, *given* invoked	≈ .656	≈ .656
correct overall (invoked *and* correct)	≈ .014	≈ .238
practiced $100 + 100$: overall errors	≈ .0073	≈ .334
after also studying $100 + 300$	unchanged	≈ .616
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).

Chris's next question is whether any of this needs EPAM in particular. Because recognition is stated for any memory, we can swap EPAM's net for ACT-R's declarative memory and change nothing else. The confident-error pattern is proved for any memory with three laws: determinacy, certainty and stability. EPAM's net has all three at every setting; ACT-R's memory has none of them, also proved. So the theorem applies to EPAM and never to ACT-R. But be careful about what that means. It doesn't mean ACT-R can't make similar errors; at this setting it answers two hundred about forty-three percent of the time. What fails is the exact pattern: ACT-R's confusion is graded, and its solver use differs from the familiar problem's. A note on the solver rows: the solver is right about sixty-six percent of the time when it's invoked; the overall figure is that times how often it's invoked. It also predicts interference: studying a new problem raises the errors on a practiced one, which EPAM never does. All the numbers are exact, at one illustrative setting.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new model **without re-running** the old one, and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that children have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; *confident* needs a further, quantitative condition.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new model **without re-running** the old one, and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that children have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; confident needs a further, quantitative condition.

So what does Chris have now? Not a simulation that happened to produce one error, but a reusable result: a set of conditions sufficient for the phenomenon, proved for every setting that meets them. It applies to a new model without re-running the old one, and through the laws it applies to any memory with those three properties. And the conditions are where the science is. Lean hasn't shown that children have sharp associations, or that a particular learning history produces the confusion; those are separate questions, theoretical or empirical. And the pattern alone doesn't make backup use rare: calling the errors confident takes the further quantitative condition we just saw.

Part 2

Working backward

What must a theory be like to guarantee a regularity?

Part two. Now Chris goes the other way: from a proposed regularity back to what a theory must be like to produce it.

Chris's conjecture

- In DOA, practice strengthens problem–answer associations; retrieval should succeed more often, and overt strategies fade.
- Chris proposes something stronger:

  Once the correct answer is the strongest association, strengthening it further never raises the probability of resorting to a backup strategy.

- Plausible, but an empirical claim about what *never* happens: hard to establish directly.
- **Robin**: first ask whether it follows from DOA.

Chris already knows that formalization exposed an ambiguity in one of the chapter's claims (next slide).

Chris's conjecture

- In DOA, practice strengthens problem–answer associations; retrieval should succeed more often, and overt strategies fade.
- Chris proposes something stronger:

  Once the correct answer is the strongest association, strengthening it further never raises the probability of resorting to a backup strategy.

- Plausible, but an empirical claim about what *never* happens: hard to establish directly.
- **Robin**: first ask whether it follows from DOA.

Chris already knows that formalization exposed an ambiguity in one of the chapter's claims (next slide).

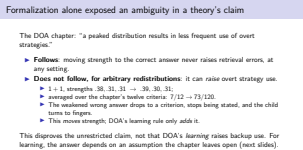
Chris's second question is about practice and strategy choice. In the distributions-of-associations model, practice strengthens the associations between problems and answers, so retrieval should succeed more often and children should rely less on overt strategies like counting. Chris proposes something stronger: once the correct answer is the strongest association, strengthening it further never raises the probability of falling back on a backup strategy. That's plausible, but it's a claim about what never happens, which is hard to establish directly from data. Robin suggests a first step: ask whether it follows from the model at all. And Chris has reason to be careful, because formalization has already exposed an ambiguity in one of the chapter's own claims.

Formalization alone exposed an ambiguity in a theory's claim

The DOA chapter: "a peaked distribution results in less frequent use of overt strategies."

- **Follows**: moving strength to the correct answer never raises retrieval errors, at any setting.
- **Does not follow, for arbitrary redistributions**: it can *raise* overt strategy use.
  - $1 + 1$, strengths $.38, .31, .31 \rightarrow .39, .30, .31$;
  - averaged over the chapter's twelve criteria: $7/12 \rightarrow 73/120$.
  - The weakened wrong answer drops to a criterion, stops being stated, and the child turns to fingers.
  - This *moves* strength; DOA's learning rule only *adds* it.

This disproves the unrestricted claim, not that DOA's *learning* raises backup use. For learning, the answer depends on an assumption the chapter leaves open (next slides).



Here's that earlier result. Formalization alone, with no composition at all, exposed an ambiguity in one of the theories, and it's one of my own. The chapter says that a peaked distribution results in less frequent use of overt strategies. Half of that follows from the model: moving strength to the correct answer never raises retrieval errors, at any setting. Half doesn't, at least for arbitrary changes to the distribution. Peaking can raise overt strategy use. Take one plus one with strengths point three eight, point three one, point three one, and move one hundredth from a wrong answer to the correct one. Averaged over the chapter's own twelve criteria, overt strategy use goes up, from seven twelfths to seventy-three over one twenty. The weakened wrong answer drops to one of the criteria, stops being stated, and the child turns to fingers. So, read as a claim about any change to the distribution, it needs a condition the chapter never stated. But notice one thing about this counterexample, because it matters: it moves strength from a wrong answer to the correct one. That's not what learning does in this model; learning adds strength without taking any from competitors. So the counterexample disproves the unrestricted claim, but it doesn't show that the model's learning raises backup use. Whether learning can do

Robin spots an ambiguity

An answer is drawn in proportion to its strength, and stated only if it **exceeds a criterion** c. Which strength?

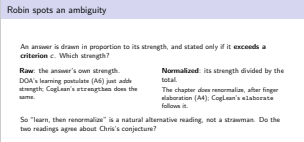
Raw: the answer's own strength.

DOA's learning postulate (A6) just *adds* strength; CogLean's `strengthen` does the same.

Normalized: its strength divided by the total.

The chapter *does* renormalize, after finger elaboration (A4); CogLean's `elaborate` follows it.

So "learn, then renormalize" is a natural alternative reading, not a strawman. Do the two readings agree about Chris's conjecture?



Robin then notices an ambiguity. In the model, an answer is drawn with probability proportional to its strength, and stated only if its strength exceeds a confidence criterion. But which strength: the raw strength, or the strength normalized by the total over all answers? This isn't a pedantic question. The chapter's learning postulate simply adds strength, and the library's learning rule does the same. But the chapter does renormalize in another place, after finger elaboration, and the library follows it there. So learning followed by renormalization is a perfectly natural alternative reading. The question is whether the two readings agree about Chris's conjecture.

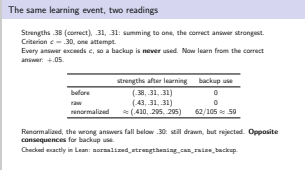
The same learning event, two readings

Strengths .38 (correct), .31, .31: summing to one, the correct answer strongest.
Criterion $c = .30$, one attempt.
Every answer exceeds c, so a backup is **never** used. Now learn from the correct answer: $+.05$.

	strengths after learning	backup use
before	$(.38, .31, .31)$	0
raw	$(.43, .31, .31)$	0
renormalized	$\approx (.410, .295, .295)$	$62/105 \approx .59$

Renormalized, the wrong answers fall below .30: still drawn, but rejected. **Opposite consequences** for backup use.

Checked exactly in Lean: `normalized_strengthening_can_raise_backup`.


The same learning event, two readings

Strengths .38 (correct), .31, .31: summing to one, the correct answer strongest.
Criterion $c = .30$, one attempt.
Every answer exceeds c, so a backup is **never** used. Now learn from the correct answer: $+.05$.

	strengths after learning	backup use
before	(.38, .31, .31)	0
raw	(.43, .31, .31)	0
renormalized	$\approx$ (.410, .295, .295)	$62/105 \approx .59$

Renormalized, the wrong answers fall below .30: still drawn, but rejected. **Opposite consequences** for backup use.
Checked exactly in Lean: `normalized_strengthening_can_raise_backup`.

They try a small example, the same one as the peaking result: the correct answer at point three eight, two wrong answers at point three one, summing to one, so the correct answer is the strongest. The criterion is point three zero and the child gets one retrieval attempt. Every answer exceeds the criterion, under either reading, so the child never uses a backup. Now the child states the correct answer and learns from it, adding point zero five. Under the raw reading the strengths are point four three, point three one, point three one: all still above the criterion, and backup use stays at zero. Under the renormalized reading, divide by the new total, one point zero five: about point four one, and two point two nine fives. Now only the correct answer clears the criterion. The wrong answers can still be drawn, but they're rejected, and with one attempt backup use jumps from zero to sixty-two over one oh five, about point five nine. Same learning event, opposite consequences. Lean checks these exact values. Chris finds the renormalized result surprising: why should strengthening an already dominant correct answer make a child more likely to abandon retrieval? Not impossible, but now there's a concrete theoretical question.

Robin overreaches

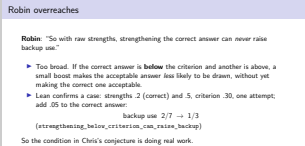
Robin: "So with raw strengths, strengthening the correct answer can *never* raise backup use."

- Too broad. If the correct answer is **below** the criterion and another is above, a small boost makes the acceptable answer *less* likely to be drawn, without yet making the correct one acceptable.
- Lean confirms a case: strengths .2 (correct) and .5, criterion .30, one attempt; add .05 to the correct answer:

$$\text{backup use } \ 2/7 \ \to \ 1/3$$

(`strengthening_below_criterion_can_raise_backup`)

So the condition in Chris's conjecture is doing real work.



Chris asks whether the raw reading guarantees the conjecture, and Robin answers too quickly: with raw strengths, strengthening the correct answer can never raise backup use. That's too broad. Suppose the correct answer is below the criterion and some other answer is above it. A small boost to the correct answer makes the acceptable answer less likely to be drawn, because it's now a smaller share of the total, without yet lifting the correct answer over the criterion. Backup use goes up. Lean confirms a concrete case: strengths point two for the correct answer and point five for another, criterion point three, one attempt. Add point zero five to the correct answer, and backup use rises from two sevenths to one third. So the condition in Chris's conjecture, that the correct answer is already the strongest, is doing real work.

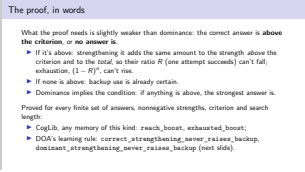
The proof, in words

What the proof needs is slightly weaker than dominance: the correct answer is **above the criterion**, or **no answer is**.

- If it's above: strengthening it adds the same amount to the strength *above* the criterion and to the *total*, so their ratio R (one attempt succeeds) can't fall; exhaustion, $(1 - R)^n$, can't rise.
- If none is above: backup use is already certain.
- Dominance implies the condition: if anything is above, the strongest answer is.

Proved for every finite set of answers, nonnegative strengths, criterion and search length:

- CogLib, any memory of this kind: `reach_boost`, `exhausted_boost`;
- DOA's learning rule: `correct_strengthening_never_raises_backup`, `dominant_strengthening_never_raises_backup` (next slide).

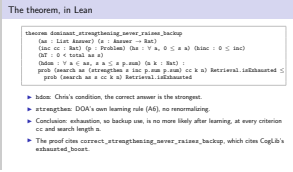


Here's the proof, in words, before the Lean. What it actually needs is a little weaker than dominance: the correct answer is already above the criterion, or no answer is. If it's above, strengthening it adds the same amount to the strength above the criterion and to the total strength. So their ratio, which is the chance that one attempt succeeds, can't go down, and the chance of exhausting retrieval, one minus that ratio to the n, can't go up. If no answer is above the criterion, backup use is already certain, so it can't rise. And dominance implies the condition: if anything clears the criterion, the strongest answer does. This is proved for every finite set of answers, every nonnegative strengths, every criterion and every search length. In the library it's stated generally, for any memory of this kind; and for the strategy-choice model it's stated for its own learning rule. The next slide shows that version.

The theorem, in Lean

```
theorem dominant_strengthening_never_raises_backup
    (as : List Answer) (s : Answer → Rat)
    (inc cc : Rat) (p : Problem) (hs : ∀ a, 0 ≤ s a) (hinc : 0 ≤ inc)
    (hT : 0 < total as s)
    (hdom : ∀ a ∈ as, s a ≤ s p.sum) (n k : Nat) :
    prob (search as (strengthen s inc p.sum p.sum) cc k n) Retrieval.isExhausted ≤
      prob (search as s cc k n) Retrieval.isExhausted
```

- `hdom`: Chris's condition, the correct answer is the strongest.
- `strengthen`: DOA's own learning rule (A6), no renormalizing.
- Conclusion: exhaustion, so backup use, is no more likely after learning, at every criterion `cc` and search length `n`.
- The proof cites `correct_strengthening_never_raises_backup`, which cites CogLib's `exhausted_boost`.

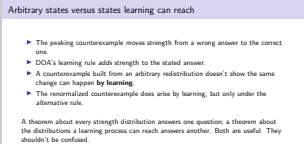


Here's the statement, verbatim. Read the hypotheses: a list of answers and their strengths; an increment and a criterion; nonnegative strengths and increment; a positive total; and h dom, Chris's condition, that every answer is at most as strong as the correct one. The conclusion compares the probability that retrieval is exhausted, which is when the backup is used, after learning with the model's own learning rule, against before, at every criterion and search length. The proof is two steps: it establishes the weaker condition from dominance and cites the general theorem for this learning rule, which in turn cites the library's theorem for any memory of this kind. That's the library working the way it should: Chris's result is a corollary of something general, and the general part is now available to the next person.

Arbitrary states versus states learning can reach

- The peaking counterexample *moves* strength from a wrong answer to the correct one.
- DOA's learning rule *adds* strength to the stated answer.
- A counterexample built from an arbitrary redistribution doesn't show the same change can happen **by learning**.
- The renormalized counterexample does arise by learning, but only under the *alternative* rule.

A theorem about every strength distribution answers one question; a theorem about the distributions a learning process can reach answers another. Both are useful. They shouldn't be confused.

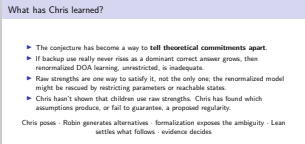


Chris also notices a difference from the peaking result. That counterexample moved strength from a wrong answer to the correct one. But the model's learning rule doesn't move strength; it adds strength to whatever answer was stated. So a counterexample built from an arbitrary redistribution doesn't show that the same change can actually happen through learning. The renormalized counterexample does arise by learning, but only under the alternative rule. The general lesson: a theorem about every possible strength distribution answers one question, and a theorem about the distributions a learning process can actually reach answers another. Both are useful, and it's easy to confuse them.

What has Chris learned?

- The conjecture has become a way to **tell theoretical commitments apart**.
- If backup use really never rises as a dominant correct answer grows, then renormalized DOA learning, unrestricted, is inadequate.
- Raw strengths are *one* way to satisfy it, not the only one; the renormalized model might be rescued by restricting parameters or reachable states.
- Chris hasn't shown that children use raw strengths. Chris has found which assumptions produce, or fail to guarantee, a proposed regularity.

Chris poses · Robin generates alternatives · formalization exposes the ambiguity · Lean settles what follows · evidence decides



So what has Chris learned? The conjecture has turned into a way of telling theoretical commitments apart. If it's true that backup use never rises as an already dominant correct answer grows stronger, then the renormalized reading of the model's learning, without further restrictions, is inadequate. Raw strengths are one way to satisfy the regularity, but not the only one: another mechanism might too, and the renormalized model might be rescued by restricting its parameters or the states it can reach. Chris hasn't proved that children use raw strengths. What Chris has done is find out which assumptions produce the regularity and which fail to guarantee it. That's reasoning backward, from a phenomenon toward possible mechanisms. Chris posed the question, Robin generated alternatives and got one wrong, formalization exposed an ambiguity, Lean settled what follows from what, and evidence will have to decide which premises describe children. And all of it is now in the library, for the next person to build on.

Part 3

Under the hood

How the composition in part 1 runs, and why its theorems transfer

Part 3

Under the hood

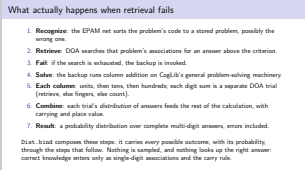
How the composition in part 1 runs, and why its theorems transfer

Part three is supporting detail: what's under the hood of the composition Chris used in part one, and why its theorems transfer.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer: correct knowledge enters only as single-digit associations and the carry rule.



First, an important detail: nothing in this model is handed the answer. Two hundred fifty for one fifty plus one hundred is assembled from single-digit trials and a carry rule, and it's never certain. So what actually happens when whole-problem retrieval fails? Step by step. The EPAM net sorts the problem's code to a stored problem, possibly the wrong one. DOA searches that problem's associations for an answer above the criterion. If the search is exhausted, the backup is invoked. The backup runs column addition on the library's general problem-solving machinery: units, then tens, then hundreds, and each column's digit sum is a separate single-digit DOA trial, with retrieval, fingers or counting. Each trial produces a distribution of possible answers, and that whole distribution feeds the rest of the calculation, with carrying and place value. The result is a probability distribution over complete multi-digit answers, errors included. The operation that does the composing is bind: it carries every possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

The general machinery, and its arithmetic instance

CogLib (`Process/ProblemSolving`), task-general:

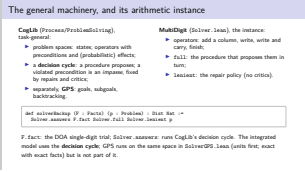
- problem spaces: states; operators with preconditions and (probabilistic) effects;
- a **decision cycle**: a procedure proposes; a violated precondition is an *impasse*, fixed by repairs and critics;
- separately, **GPS**: goals, subgoals, backtracking.

MultiDigit (`Solver.lean`), the instance:

- operators: add a column, write, write and carry, finish;
- `full`: the procedure that proposes them in turn;
- `lenient`: the repair policy (no critics).

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
```

`F.fact`: the DOA single-digit trial; `Solver.answers`: runs CogLib's decision cycle. The integrated model uses the **decision cycle**; GPS runs on the same space in `SolverGPS.lean` (units first; exact with exact facts) but is not part of it.



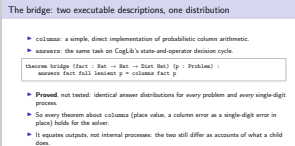
Here's the machinery underneath. The library's problem-solving module is task-general. It has a problem space, with states and operators that have preconditions and effects, and an effect can be a probability distribution. It has a decision cycle: a procedure proposes an operator; if its precondition is violated, that's an impasse, and repair heuristics and critics fix it. And, separately, it has GPS-style means-ends analysis, with goals, subgoals and backtracking. Multi-digit addition is one instance: operators to add a column, write, write and carry, and finish; a procedure, full, that proposes them in turn; and a repair policy, lenient. The backup is one call: answers, which runs the library's decision cycle, given the single-digit DOA trial, the procedure and the repair policy. One precision: the integrated model uses the decision cycle. GPS runs on the same problem space in a separate file, where it's proved to work units first and to be exact with exact facts, but it isn't part of the integrated model.

The bridge: two executable descriptions, one distribution

- `columns`: a simple, direct implementation of probabilistic column arithmetic.
- `answers`: the same task on CogLib's state-and-operator decision cycle.

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

- **Proved**, not tested: identical answer distributions for *every* problem and *every* single-digit process.
- So every theorem about `columns` (place value, a column error as a single-digit error in place) holds for the solver.
- It equates *outputs*, not internal processes: the two still differ as accounts of what a child does.



Now the bridge. There are two executable descriptions of column addition. One, columns, is a simple direct implementation of probabilistic column arithmetic. The other, answers, does the same task on the library's general state-and-operator decision cycle. The bridge theorem says they produce identical answer distributions, for every problem and for every single-digit process you plug in. That's proved, not established by testing examples. Its payoff is that every theorem proved about the simple version, like place value, or a column error being a single-digit error moved to its place, holds for the solver too. But be precise about what it equates: outputs, not internal processes. As accounts of what a child does, the two are still different. This is the library idea in miniature: results transfer between independently defined mechanisms.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

Three kinds of reuse make the composition deductive rather than just executable. A bridge theorem says the solver on the decision cycle answers exactly as a fixed column procedure does, for every problem and every single-digit process, so everything proved about the fixed procedure transfers. Associativity: every arrow is a two-stage draw of distributions, and draws regroup, so a theorem about a component holds where it sits inside the whole. And well-formedness is inherited: that the composed trial is a genuine probability distribution, total mass one and no negative weights, is proved by chaining lemmas from three packages. None of this is psychology. It's what lets the psychology transfer.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

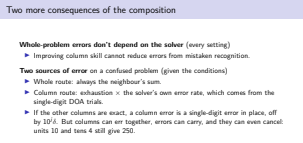
Two more consequences of the composition

Whole-problem errors don't depend on the solver (every setting)

- Improving column skill cannot reduce errors from mistaken recognition.

Two sources of error on a confused problem (given the conditions)

- Whole route: always the neighbour's sum.
- Column route: exhaustion $\times$ the solver's own error rate, which comes from the single-digit DOA trials.
- If the other columns are exact, a column error is a single-digit error in place, off by $10^j\delta$. But columns can err together, errors can carry, and they can even cancel: units 10 and tens 4 still give 250.



Two more consequences. First, whole-problem errors don't depend on the solver at all, at any setting. Psychologically that's a prediction about intervention: teaching column skill can't reduce errors that come from mistaken recognition; only learning that changes recognition or associations can. Second, a confused problem has two sources of error. The whole route always gives the neighbour's sum. The column route errs with probability equal to exhaustion times the solver's own error rate, which comes entirely from the single-digit trials. When only one column errs, the error is that single-digit error moved to its place. But that's a special case: columns can err together, errors propagate through carries, and they can even cancel. A units result of ten and a tens result of four still give two hundred fifty.

One memory for both levels

Column facts from the *same* memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`memFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

One memory for both levels

Column facts from the same memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`sameFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

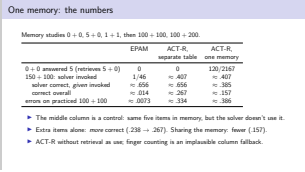
So far the solver's single-digit facts came from their own table, separate from the memory that recognizes whole problems. A child has one memory, so we removed the separation. Each digit pair, say five plus zero, is coded as a problem in the same code space, recognized by the same memory, and retrieved from the same strength table, with fingers and counting if retrieval fails. Three things follow. First, if a pair is recognized as itself, its fact is exactly the old DOA fact, so the old model is a special case and every earlier theorem carries over. Second, a pair taken for another pair gives confident errors one level down, by the same general theorem, with no new proof. Third, a caution: the old noninterference theorem held the solver fixed while the net learned. With one memory, learning could change the solver too, so noninterference needs a condition: it holds when the problem's digit pairs were themselves learned, and that we prove.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	120/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.



Here are the numbers, with the memory holding three digit pairs and the two problems. EPAM recognizes every pair as itself, so nothing changes from before. ACT-R confuses pairs: zero plus zero sometimes retrieves five plus zero and is answered five. Two things changed at once for ACT-R, the memory holds more items and the solver uses it, so the middle column is a control: the same five items in memory, but the solver keeps its separate table. The extra items alone actually raise correct answers a little, from about twenty-four to twenty-seven percent, because more problems reach an accurate solver. It's the solver's use of the memory that brings them down to about sixteen: the solver's accuracy when invoked drops from sixty-six percent to under forty. Two caveats: this is ACT-R without retrieval changing memory, and finger counting is an implausible fallback for someone doing three-digit addition.

Part 4

What this shows, and what it doesn't

Checks, kinds of results, limits

Finally, what all this shows and doesn't: how results are labeled, how we checked the formalization, where it sits, and its limits.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	$45/46$, $1/46$, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	45/46, 1/46, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

For credibility, it's essential to say what each result rests on. Some are library facts, like the bridge and well-formedness: needed, but not psychology. Some are compositional deductions that hold at every setting, like solver independence. Some are conditional, like confident errors, and the conditions are where the science is: the proofs say exactly which premise a failed test would implicate. And some are just numbers at chosen settings. None of these is yet an empirical finding about people, and the numbers are illustrations, not fits to children's data.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Lean checks that the theorems follow from the definitions, not that the definitions say what the prose model says. So we checked that link independently. A prose specification with no Lean in it, checks preregistered before any simulator existed, and a fresh AI agent that wrote a simulator from the prose alone. Every check whose hypothesis arose passed, and two hundred forty-three outcome frequencies matched Lean's exact values. And it found things the proofs didn't. Five theorems had hypotheses that never arose in the learning histories we tested; that doesn't show they're unreachable, only that the theorems said little about those nets. There's a gap at zero total strength. And a predicted learning sequence turns out to depend on an assumption the model never stated: the initial associations of a newly discriminated problem.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Where does this sit? There's a lot of prior formal work on cognitive architectures: a specification of Soar in Z, the Sceptic specification language, formal semantics and model checking for ACT-R, and recently Soar models compiled for the PRISM probabilistic model checker. It helps to separate three approaches. Simulation samples executions. Probabilistic model checking analyzes a specified model exhaustively; that's real formal verification, not simulation, and it can treat parameters symbolically. Deductive proof establishes results for whole classes of models that satisfy theoretical assumptions. So the contrast isn't simulation versus verification; it's model verification versus theoretical deduction, and they're complementary. CogLean doesn't introduce formal methods to cognitive science. What's new is the combination: a shared, cumulative library with reusable proofs, composition across theories, and deductive discovery.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present example composes locally, for one task, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present example composes locally, for one task, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

And to be clear about cognitive architectures. CogLean is complementary to Soar, ACT-R and the Common Model of Cognition, not an alternative to them. Architectures accumulate theory as shared mechanisms in which models are built. CogLean tries to accumulate proved results about those mechanisms, including results over whole classes of them, and nothing stops an architecture from carrying such results itself. The confident-errors theorem is the example: it holds for every memory that satisfies three laws, which EPAM's memory has and ACT-R's has none of. That's also the limit of what's shown: the laws are sufficient, not shown to be necessary, and nothing here establishes general equivalence or divergence between architectures. It's a possible formal foundation for expressing, composing and reasoning about their mechanisms and architectural commitments. The present example composes mechanisms locally, for one task, without committing to any general architecture. The longer-term aim is to formalize architectural constraints themselves, so that their implications can be proved and alternative architectures compared within one shared framework.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

A word on what proofs do and don't show. Lean guarantees validity. It doesn't guarantee that the premises describe people, and it doesn't guarantee explanation, because the premises might already contain the phenomenon. We had exactly that case: a serial-position result whose mechanism was a bowed schedule, so proving a bowed curve was valid and nearly empty. It's now derived from an attention rule over anchors, which also gives the isolation effect, a second consequence of the same mechanism. The library keeps an explicit audit that sorts its results as definitional, close, derived or compositional. That's a judgment, written down in the open.

What CogLean is, and isn't

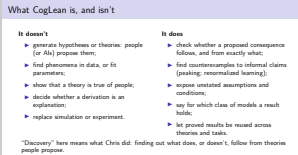
It doesn't

- generate hypotheses or theories: people (or AIs) propose them;
- find phenomena in data, or fit parameters;
- show that a theory is true of people;
- decide whether a derivation is an explanation;
- replace simulation or experiment.

It does

- check whether a proposed consequence follows, and from exactly what;
- find counterexamples to informal claims (peaking; renormalized learning);
- expose unstated assumptions and conditions;
- say for which class of models a result holds;
- let proved results be reused across theories and tasks.

"Discovery" here means what Chris did: finding out what does, or doesn't, follow from theories people propose.



So, to be clear about what this is and isn't. CogLean doesn't generate hypotheses or theories; people, or AIs, propose them. It doesn't find phenomena in data or fit parameters. It doesn't show that a theory is true of people, and it can't decide whether a derivation is an explanation; that's a scientific judgment. And it doesn't replace simulation or experiment. What it does is check whether a proposed consequence actually follows, and from exactly which premises. It finds counterexamples to informal claims, like the peaking claim. It exposes assumptions and conditions that theories leave unstated. It says for which class of models a result holds. And it lets proved results be reused across theories and tasks. When I say discovery, that's what I mean: finding out what does, or doesn't, follow from theories that people propose.

Chris and Robin: from reasoning to model construction

A future possibility, not a present accomplishment.

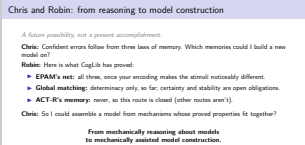
Chris: Confident errors follow from three laws of memory. Which memories could I build a new model on?

Robin: Here is what CogLib has proved:

- **EPAM's net:** all three, once your encoding makes the stimuli noticeably different.
- **Global matching:** determinacy only, so far; certainty and stability are open obligations.
- **ACT-R's memory:** never, so this route is closed (other routes aren't).

Chris: So I could assemble a model from mechanisms whose proved properties fit together?

From mechanically reasoning about models
to mechanically assisted model construction.



A last turn of the story, and a possibility, not something built. Once a theorem holds for every memory that satisfies stated laws, you can turn the question around: which mechanisms in the library meet the requirements of a result you want your model to have? Chris asks which memories could support confident errors, and Robin can answer from the theory matrix, where we did this search by hand. EPAM's net has all three laws, but certainty needs noticeably different stimuli, a property of the encoding that the rest of Chris's model must guarantee: an obligation handed to another component. Global matching has determinacy, but certainty and stability are still open. ACT-R's memory provably never has them, so this route is closed for it, though not every route to confident errors. A failed obligation doesn't show that no suitable model exists, and the laws need hold only in states the model can reach. With enough mechanisms, laws and theorems, the library could guide the construction of models, not just analyze them. Lean gives us proof checking, automation and instance search; the organization and search machinery don't exist yet. None of this invents theories or makes a model psychologically plausible: the matrix judges that people lack certainty, so the one memory that fits isn't a good model of people. The direction:

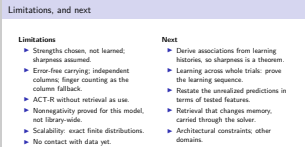
Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.



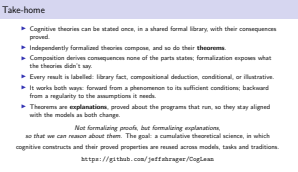
The limitations are real. The associative strengths are chosen, not learned, and sharpness, the key premise of confident errors, is assumed. Carrying is error-free and columns are independent, and the columns fall back on finger counting, which no one doing three-digit addition would do. ACT-R is represented by its retrieval rule only: retrieving something doesn't change its memory, so all the ACT-R numbers are for that version. Nonnegative weights are proved for this model but not yet across the library. Exact finite distributions raise scalability questions. And there's no contact with data yet. Next: derive the associations from learning histories, so that sharpness becomes a theorem instead of an assumption; add learning across whole trials, so the predicted learning sequence can be proved; restate the predictions whose hypotheses weren't realized; let retrieval change memory, carrying the memory's state through the solver; and move on to architectural constraints and other domains.

Take-home

- Cognitive theories can be stated once, in a shared formal library, with their consequences proved.
- Independently formalized theories compose, and so do their **theorems**.
- Composition derives consequences none of the parts states; formalization exposes what the theories didn't say.
- Every result is labelled: library fact, compositional deduction, conditional, or illustrative.
- It works both ways: forward from a phenomenon to its sufficient conditions; backward from a regularity to the assumptions it needs.
- Theorems are **explanations**, proved about the programs that run, so they stay aligned with the models as both change.

Not formalizing proofs, but formalizing explanations, so that we can reason about them. The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`



To sum up. Cognitive theories can be stated once, in a shared formal library, with their consequences proved rather than simulated. Independently formalized theories compose, and so do their theorems. Composition derives consequences that none of the parts states, and formalization exposes things the theories never said. It works in both directions, as Chris's two questions showed: forward from a phenomenon to conditions sufficient for it, and backward from a proposed regularity to the assumptions it needs. And every result is labelled: a library fact, a compositional deduction, a conditional result, or an illustration. The goal is a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and research traditions, the way mathematical results are reused in Mathlib. One example doesn't establish that. It shows that the approach can produce results of the right kind. The point is not merely to run cognitive models, but to reason about them. CogLean isn't primarily about formalizing proofs. It's about formalizing explanations so that we can reason about them, and using Lean to keep those explanations rigorously aligned with the computational models they explain.